

```
(burnies) ~$ conda env list
# conda environments:
#
* burnies          /home/.../miniconda2/envs/burnies
gl-env            /home/.../miniconda2/envs/gl-env
hat               /home/.../miniconda2/envs/hat
nd                /home/.../miniconda2/envs/nd
nc2               /home/.../miniconda2/envs/nc2
openenv           /home/.../miniconda2/envs/openenv
p1                /home/.../miniconda2/envs/p1
py                /home/.../miniconda2/envs/py
root              /home/.../miniconda2
```

Figure 4: List of environments

available in the PyPI repository.

The following command will uninstall the package and its dependencies from the environment, *env_name*:

```
conda remove --name env_name package
```

The following command will deactivate the environment in Linux based systems:

```
source deactivate
```

At the time of creation of an environment, the command to deactivate it will also be displayed to the user. It is unnecessary to specify the name of the environment; the current environment will be automatically deactivated by this command.

The following command will delete the environment and the packages associated with it:

```
conda remove --name env_name --all
```

List of packages in an environment

To list the packages installed in a particular environment, use the following command:

```
conda list --name env_name
```

If you omit *--name env_name*, packages in the current environment will be listed.

List of environments

Two commands can be used to get the list of environments. These are:

```
conda info --env
```

...or:

```
conda env list
```

The current environment is distinguished from other environments with a '*' as shown in Figure 4.

Copy an environment

It is possible to copy an environment from one system to

another. We can export the configuration of the current environment into a *.yaml* file using the following command:

```
conda env export > file_name.yaml
```

The *.yaml* file generated can be copied to any number of systems. The following code shows the content of a *.yaml* file generated when the configuration of an environment *f1*, which does not have any packages installed in it, is exported to a file:

```
name: f1
channels:
- defaults
prefix: C:\Users\admin\Miniconda3\envs\f1
```

The configuration of another environment in which *pip* is installed looks as follows:

```
name: n1
channels:
- defaults
dependencies:
- pip=9.0.1=py36_1
- python=3.6.1=0
- setuptools=27.2.0=py36_1
- vs2015_runtime=14.0.25123=0
- wheel=0.29.0=py36_0
prefix: C:\Users\admin\Miniconda3\envs\n1
```

Packages other than *pip* in the dependencies list are the dependencies of *pip* that are automatically installed. Once the file is copied to the destination system, execute the following command to create an environment exactly similar to the one in the source system:

```
conda env create -f file_name.yaml
```

A folder named *n1*, which is the name of the environment as specified in the environment file, will be created in the destination system in the path *miniconda3/envs*. When you execute the above command in the destination system, make sure that your current directory is Miniconda3.

YML file

YAML is an Ansible configuration management language. Every YAML file is organised as a list or record containing one or more members. All the members in a list will begin with a '-' (hyphen followed by a single space). A dictionary will be arranged as a 'key: value' pair (the colon must be followed by a space). Members of the same list or record will be arranged with the same indentation. This much detail is enough for creating an environment file. More details about YAML syntax can be found at <http://docs.ansible.com/ansible/YAMLSyntax.html>.